Hosting Plan — AI Customer Support Automation Platform

Host Site

All components are deployed on **Microsoft Azure**.

Component	Platform	Service
Frontend (React)	Microsoft Azure	Azure App Service (Node.js 20.x)
Auth Service (Spring Boot)	Microsoft Azure	Azure App Service (Java 21)
AI Service (FastAPI)	Microsoft Azure	Azure App Service B1 SKU — ai-customersupport.azurewebsites.net
Database	Microsoft Azure	Azure Database for MySQL Flexible Server — ai-support-db.mysql.database.azure.com
Vector Store (ChromaDB)	Microsoft Azure	Bundled inside AI Service container (no separate service needed)

Deployment Strategy

Step 1 — Database Setup

- Provision **Azure MySQL Flexible Server** from the Azure Portal.
- Store DB credentials as Azure App Service Environment Variables (not in code).

Step 2 — Deploy AI Service

- GEMINI_API_KEY is added as an **Azure Application Setting** (environment variable).
- ChromaDB and knowledge-base files are **bundled directly** in the deployment package.

Step 3 — Deploy Auth Service

- Service runs on **port 80** (Azure App Service default).

Step 4 — Deploy Frontend

- Served via a server on the **PORT 8080 (default).**

Security

Authentication

- JWT tokens (HS256, 24-hour expiry) issued by Spring Security.
- BCrypt hashing for all user passwords.
- Every protected API endpoint validates the JWT before processing.

Database Security

- DB credentials are stored as Azure Application Settings, never committed to GitHub**.**

CORS Policy

- Currently set to allow_origins=["*"] for development.
- Cab be restricted to the specific frontend Azure domain: allow_origins=["https://<frontend>.azurewebsites.net"]

How End Users Access the Services

End users access the entire platform through a standard web browser

They simply navigate to the React frontend URL hosted on Azure App Service over HTTPS.

Complete Access Flow (All Components)

Step 1 — User opens browser and navigates to the React frontend URL (Azure App Service).

Step 2 — Frontend serves the React SPA (single-page application) — Login/Register page is displayed.

Step 3 — User submits credentials → Frontend calls Auth Service POST /api/auth/register or POST /api/auth/login.

Step 4 — Auth Service validates input → BCrypt password check → queries Azure MySQL for user record via JDBC/HikariCP (SSL, port 3306).

Step 5 — Auth Service generates a signed **JWT (HS256, 24-hour expiry)** and returns it to the Frontend.

Step 6 — Frontend stores the **JWT** in localStorage and unlocks the main application UI.

Step 7 — User types a support query in the **AI Chat UI →** Frontend sends **POST /api/ai/query with Authorization: Bearer <token>** header to the AI Service.

Step 8 — AI Service validates the **JWT →** passes the query to the **RAG Engine.**

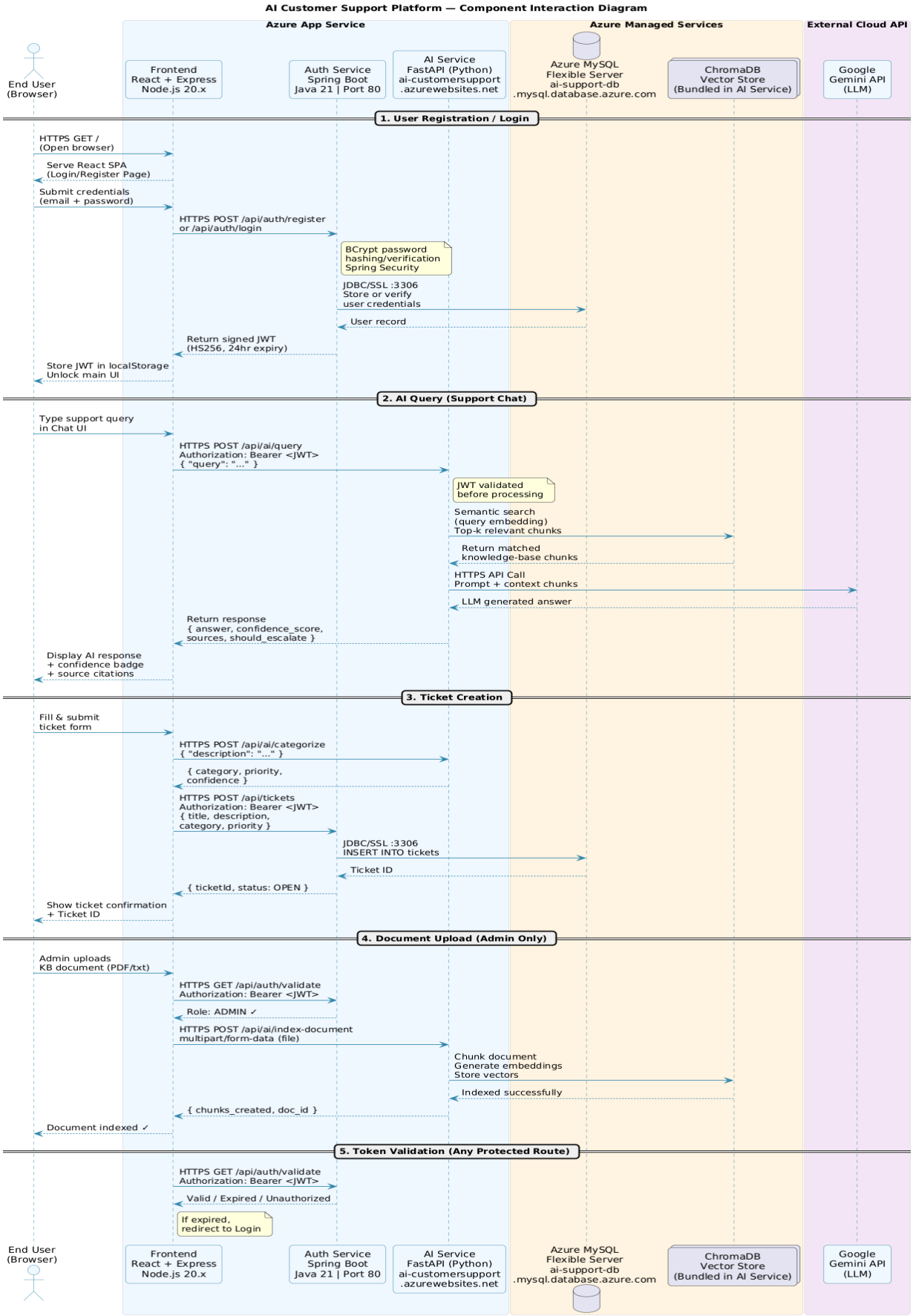
Step 9 — RAG Engine queries **ChromaDB** (bundled vector store) for semantically relevant knowledge-base chunks.

Step 10 — RAG Engine sends the query **+** retrieved chunks as a prompt to **Google Gemini API (external HTTPS call).**

Step 11 — Gemini API returns the **LLM-generated** answer **→ AI Service** calculates a confidence score and escalation flag.

Step 12 — AI Service returns the response to the **Frontend →** displayed to the user with **confidence indicator and source citations.**

Component Interaction Diagram:



Deployment & Component Diagram: